

DEEP LEARNING

Report 4: Neural Network

Student: Vu Ha Vy – 2540112

Implementation

Sigmoid activation function

Each neuron applies the sigmoid activation function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where z is the linear combination between the inputs and weights.

```
def sigmoid(z):
    return 1.0 / (1.0 + math.exp(-z))
```

Neuron implementation

A neuron stores its weights and bias. During feedforward, the neuron computes the weighted sum:

$$z = \sum_{i=1}^n x_i w_i + b$$

and then applies the sigmoid activation:

$$a = \sigma(z)$$

```
class Neuron:
    def __init__(self, weights, bias):
        self.weights = weights
        self.bias = bias

    def feedforward(self, inputs):
        z = 0
        for i in range(len(inputs)):
            z += inputs[i] * self.weights[i]
        z += self.bias

        a = sigmoid(z)
        return a
```

Layer implementation

A layer is composed of several neurons. The outputs of all neurons are stored in a list and returned as the output of the layer.

```
class Layer:
    def __init__(self, neurons):
        self.neurons = neurons

    def feedforward(self, inputs):
        outputs = []

        for neuron in self.neurons:
            output = neuron.feedforward(inputs)
            outputs.append(output)

        return outputs
```

Neural network implementation

The neural network is implemented as a sequence of layers. The output of one layer becomes the input of the next layer. For each layer:

$$z^{(l)} = (W^{(l)})^T a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = \sigma(z^{(l)})$$

```
class NeuralNetwork:
    def __init__(self, layers):
        self.layers = layers

    def feedforward(self, inputs):
        outputs = inputs

        for layer in self.layers:
            outputs = layer.feedforward(outputs)

        return outputs
```

Neural Network Architecture

The architecture of the neural network is read from the file `nn.txt`. The first line contains the number of layers. The following lines contain the number of neurons in each layer. For this experiment:

- Input layer: 2 neurons
- Hidden layer: 2 neurons
- Output layer: 1 neuron

The network is initialized using object oriented programming. Each layer contains neurons, and each neuron contains weights and bias. The weights and bias are initialized randomly in the interval $[0, 1]$.

```
def build_nn(layer_sizes):
    layers = []

    for i in range(1, len(layer_sizes)):
        n_inputs = layer_sizes[i - 1]
        n_neurons = layer_sizes[i]

        neurons = [
            Neuron(
                weights=[random.uniform(0, 1)
                        for _ in range(n_inputs)],
                bias=random.uniform(0, 1)
            )
            for _ in range(n_neurons)
        ]

        layers.append(Layer(neurons))

    return NeuralNetwork(layers)

neural_network = build_nn(layer_sizes)
```

XOR Experiment

The XOR problem cannot be solved using a single logistic regression classifier because the data is not linearly separable. The XOR truth table is:

x_1	x_2	$x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

Table 1: Truth table of XOR

The hidden layer is designed using two intermediate logical operations:

- $\text{NOT}(x_1 \text{ AND } x_2)$
- $x_1 \text{ OR } x_2$

The output layer computes:

$$\text{XOR} = \text{NOT}(x_1 \wedge x_2) \wedge (x_1 \vee x_2)$$

The weights are initialized exactly as in the lecture slides.

```
neural_network.layers[0].neurons[0].weights = [-1, -1]
neural_network.layers[0].neurons[0].bias = 1.5

neural_network.layers[0].neurons[1].weights = [1, 1]
neural_network.layers[0].neurons[1].bias = -0.5

neural_network.layers[1].neurons[0].weights = [1, 1]
neural_network.layers[1].neurons[0].bias = -1.5
```

Results

The network is tested with the four possible XOR inputs.

```
data = [[0,0], [0,1], [1,0], [1,1]]

for i in data:
    output = neural_network.feedforward(i)
    print(i, ":", round(output[0], 5))
```

The obtained outputs are:

Input	Expected Output	Network Output	Predicted Class
(0, 0)	0	0.42436	0
(0, 1)	1	0.43657	0
(1, 0)	1	0.43657	0
(1, 1)	0	0.42436	0

Table 2: Results of the XOR neural network

Using the decision threshold 0.5, all four outputs are classified as 0. Therefore, the network gives the correct result for (0, 0) and (1, 1), but gives the wrong result for (0, 1) and (1, 0). This is because the sigmoid activation function does not return exact boolean values 0 and 1. For example, for input (0, 1), the two hidden neurons return approximately 0.62246 and 0.62246. Although (0, 1) should belong to class 1, the final output is still below 0.5, so it is classified as 0. The output neuron is computed by:

$$z = 0.62246 + 0.62246 - 1.5 = -0.25508$$

$$\hat{y} = \sigma(-0.25508) = 0.43657$$